

实验 5 Linux 进程间通信—软中断信号处理

1. 实验目的

基本掌握 Linux 进程软中断通信的基本原理和实现方法。

2. 相关知识与系统调用

软中断信号的处理，主要是实现同一用户的各进程之间的通信。

(1) kill(pid, sig): 发送信号

(2) signal(sig, func): 指定进程对信号 sig 的处理行为是调用函数 func。

3. 实验程序

【程序 3_13】使用系统调用fork()创建两个子进程，让父进程分别与两个子进程进行软中断通信。

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <stdlib.h>
4. #include <sys/wait.h>
5.
6. int wait_mark;
7.
8. void waiting(){                //自定义 waiting 函数，与 wait(0)作用不同，其作用是
    通过循环是子程序停止。
9.     while(wait_mark != 0);
10. }
11.
12. void stop(){                  //调用此函数时，使 wait_mark = 0,结束 waiting 函数
    种得循环，使子进程继续执行。
13.     wait_mark = 0;
14. }
15.
16. void main(){
17.     int p1,p2;
18.     while((p1 = fork()) == -1);
19.     if( p1 > 0){
20.         while((p2 = fork()) == -1);
21.         if(p2 > 0){
22.             printf("parent\n");
23.             sleep(1);
24.             kill(p1,16);
25.             kill(p2,17);
26.             wait(0);
27.             wait(0);
```

```

28.         printf("parent process is killed!\n");
29.         exit(0);
30.     }
31.     else{
32.         printf("p2\n");
33.         wait_mark = 1;
34.         signal(17,stop);      //将信号 17 和函数 stop 关联起来
35.         waiting();            //循环，直到父进程发送信号 17 时，调用 stop 结束
                                循环继续执行
36.         printf("child process 2 is killed by parent!\n");
37.         exit(0);
38.     }
39. }
40. else{
41.     printf("p1\n");
42.     wait_mark = 1;
43.     signal(16,stop);          //将信号 16 和函数 stop 关联起来
44.     waiting();                //循环，直到父进程发送信号 16 时，调用 stop 结束
                                循环继续执行
45.     printf("child process 1 is killed by parent!\n");
46.     exit(0);
47. }
48. }

```

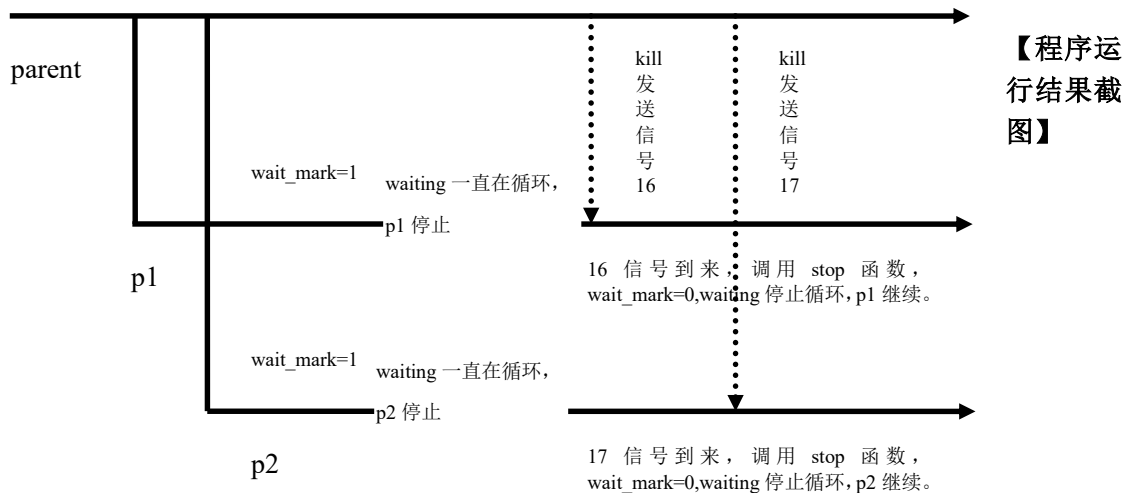


图 3.3 程序执行过程图

```

p1
parent
p2
child process 1 is killed by parent!
child process 2 is killed by parent!
parent process is killed!

```

4. 实验思考与扩充

- (1) 程序运行结果为何不唯一？除了上述情况外，是否还有其他情况？
- (2) 程序中 `signal()` 的作用是什么？
- (3) 如何修改【程序 3_13】，当用户按 `ctrl+c` 键时，父进程能够捕捉到它对应的中断信号，然后再向两个子进程发信号？
- (4) 程序中 `sleep(1)` 的作用是什么？